

Introducción a los IDEs. Eclipse. IntelliJ IDEA. Otros

Índice

Introducción a los IDE	2
¿Qué hace que un IDE sea un IDE?	4
Los IDEs más populares. Eclipse. NetBeans. IDEA	5
Eclipse	5
NetBeans	6
IntelliJ IDEA	7

Introducción a los IDE

IDE es el acrónimo en inglés de Integrated Development Environment o, en castellano, Entorno de Desarrollo Integrado. Un IDE es una aplicación que provee un montón de facilidades y herramientas para programadores orientadas al desarrollo de software. Un IDE normalmente consta de al menos un editor avanzado de texto, integración con el compilador o la herramienta de construcción y/o empaquetado y un depurador de código. Algunos IDEs van más allá, e introducen manejo de versiones o integración con sistemas de versionado, un constructor automatizado de interfaces de usuario, herramientas de generación de código, herramientas de generación de diagramas para diseño orientado a objetos, herramientas para bases de datos, etc.

Los IDEs están pensados para mejorar la productividad del desarrollador, integrando varias herramientas (como las mencionadas anteriormente) dentro de una misma interfaz gráfica. Es decir, en vez de tener un editor de código, una consola de base de datos, una consola para el manejo de versionado, etc. Cada uno con su interfaz gráfica particular diseñada por diferentes proveedores, el IDE intenta tener todo bajo un mismo ambiente (integrado) de desarrollo. Esto ayuda a reducir el tiempo necesario para configurar todas las herramientas para comenzar el desarrollo de un sistema y achata la curva de aprendizaje que pueden tener múltiples herramientas, dado que se debe manejar una única herramienta apta todas las necesidades de desarrollo. Otras características de los IDEs es que ayudan a la productividad, no solo por agrupar múltiples herramientas en una, sino por las facilidades que estas brindan. Una de las más importantes es estar continuamente analizando el código fuente a medida que se escribe, compilando en tiempo real y reportando errores y/o posibles situaciones problemáticas e informándolas automáticamente. Sin un IDE deberíamos valernos únicamente de un compilador, que se corre luego de haber escrito el código y los errores o problemas aparecerían solo luego de haberlo escrito. Mediante un IDE podemos empaquetar con simples operaciones los entregables, también reduciendo el tiempo necesario para armarlos. Por último, y quedando otras cosas para destacar, debe mencionarse la capacidad de depurar o “debuggear” el código. Los IDEs tienen la fantástica capacidad de “suspender” la ejecución

en puntos específicos del código, ejecutar línea por línea, ejecutar código condicionalmente, analizar los valores de las variables locales, las variables miembro, tener un pantallazo del estado del sistema, etc., sin salir de la herramienta. Anteriormente, no era tarea fácil y por lo general se necesitaba de herramientas extras. Aquellos IDEs dedicados específicamente a un lenguaje pueden agregar herramientas muy específicas a ese lenguaje incrementando aún su utilidad y la productividad del desarrollador.

Inicialmente, solo se utilizaban los editores de texto para escribir el código fuente mediante una consola o terminal (y si vamos a un período anterior, ni siquiera se “escribía el código”, ya que no había terminal y se ingresaba todo el programa mediante tarjetas perforadas). Luego de unos años, aparecieron los primeros “editores avanzados” que, además de comandos de teclado (teclas de función en el teclado para grabar, abrir un archivo, mandar a compilar, etc.), empezaban a marcar con diferente tipografía las palabras clave (no necesariamente colores, dependiendo de las capacidades gráficas de la plataforma). Los ejemplos más comunes son Vim o Emacs, aún vigentes hoy en día. Con el advenimiento de las interfaces gráficas, los editores avanzados se volvieron muy fáciles de utilizar, agregando más funcionalidades (integración con el sistema operativo, capacidades avanzadas de entrada/salida, autocompletado básico –cerrar etiquetas/bloques de código– y algunas otras pequeñas ayudas) y comenzaron a aparecer los primeros IDEs. A modo anecdótico, podemos citar a BASIC como el primer lenguaje en proveer un “IDE” (que era más bien un editor avanzado, integrado con el compilador, depurador, etc.) allá por el año 1989. Incluso antes, en el 74, IBM lanzó algo similar a un IDE llamado Structured Programming Facility. Durante los últimos años, aparecieron muchos IDEs de propósito general o específicos de un lenguaje. En cuanto a JAVA, JBuilder (originalmente de Borland) supo ser uno de los más completos y populares. Visual Basic 6.0, recordadísimo IDE para el lenguaje de igual nombre, que ayudó a construir incontables sistemas apareció en 1991 y duró hasta 1998. Eclipse, que existe desde el 2001, comenzó como un IDE para Java apoyado por IBM y fue abriéndose para soportar múltiples lenguajes: C, C++, PHP, Perl y muchísimos más. En los años más recientes, aparecieron muchos IDEs para JAVA; los más notables son NetBeans (con su espectacular

diseñador de interfaces gráficas swing) e IntelliJ IDEA. Es importante mencionar que hoy en día está ganando mucho terreno el VisualStudio Code (no tanto para JAVA, pero es uno de los lenguajes soportados). Aun así, Eclipse sigue siendo el más popular y prácticamente el estándar de mercado, especialmente porque es de código abierto, por tanto, gratis y sin que eso imponga limitaciones en su funcionalidad (como puede ser un IDE pago como IntelliJ IDEA, cuya versión gratuita es limitada).

Como referencia, se incluye acá un enlace de Wikipedia, con la lista completa de IDEs y sus características (inglés):

https://en.wikipedia.org/wiki/Comparison_of_integrated_development_environments#Java

¿Qué hace que un IDE sea un IDE?

Más o menos, el consenso general es que un IDE tenga las siguientes características:

1. **Edición avanzada de texto:** todo lo que debe tener un editor de texto (sin poder controlar el formato, ya que es irrelevante en este caso), como deshacer, rehacer, buscar, reemplazar, etc. Algunos IDEs agregan algo extra como selección múltiple, selección por columnas, búsqueda entre múltiples archivos, búsqueda con expresiones regulares, etc.
2. **Resaltado de la sintaxis:** las palabras claves se remarcan de alguna manera (tipografía, colores) para indicar que están correctamente escritas o utilizadas.
3. **Análisis y/o compilación continua del código:** presentando los problemas en una lista o con sobreimpresos dentro del área de escritura del código.
4. **Autocompletado:** ya sea de bloques de código (por ejemplo, al abrir un bloque con una llave, automáticamente se cierra, para evitar que el programador se olvide y luego tenga un error de sintaxis) o de instrucciones o de llamadas a variables y/o funciones. En todos los casos, puede ser automático o mediante alguna combinación de teclas (normalmente TAB o CTRL+espacio).

5. **Vista de los archivos del proyecto:** un pantallazo del árbol de archivos y directorios que constituyen el proyecto.
6. **Refactoring:** se refiere a la capacidad de cambiar partes del código y que automáticamente se cambien todas las partes dependientes dentro del proyecto (es decir, de cambiar el nombre de una función, automáticamente se cambia en todos los lugares donde es llamada).
7. **Depuración:** la depuración del código debe estar integrada. El IDE debe dar la posibilidad de ejecutar el código paso a paso para poder buscar errores y depurar diferentes partes del programa.
8. **Integración con herramientas de la plataforma o el sistema operativo:** el IDE debe permitir poder llamar a las herramientas del sistema “desde adentro”. Esto incluye llamada a comandos u otras herramientas, pudiéndolas configurar y/o asignarle alguna combinación de teclas para su ejecución.
9. **Integración con un sistema de versionado de código.**
10. **Opcional – estructura de plugins:** prácticamente todos los IDEs pueden “extenderse” con plugins de todo tipo, ya sea para soportar lenguajes adicionales, agregar ayudas al desarrollador, integración con bases de datos, etc.

Los IDEs más populares. Eclipse. NetBeans. IDEA

Eclipse

Es hoy el más popular. Como muchos, Eclipse mantiene un “modelo” de proyecto, sabiendo donde está el código fuente donde deben arrojar los compilados y así como correrlos, donde están las librerías y, qué código se ejecutó y un historial de ejecuciones, etc. Eclipse tiene la característica de ser un IDE “poliglota”, pudiendo obtenerse diferentes “distribuciones” preparadas para distintos lenguajes o incluso distintas tareas dentro de un mismo lenguaje (Java WEB o Java Desktop, por ejemplo). Cada distribución tiene una serie de herramientas preinstaladas, sin limitar que se puedan agregar más o

reemplazarlas por otras (es decir, puedo usar la distribución para desktop y luego agregar todas las herramientas que necesite para trabajar con web).

Eclipse tiene todas las características básicas mencionadas, más otras adicionales, como ser búsquedas indexadas de los archivos de código fuente y otros archivos, búsquedas dedicadas a diferentes tipos de archivos (si son de configuración o de código, por ejemplo), herramientas de testeo, de análisis de performance, de cobertura de código, etc. Eclipse mantiene diferentes maneras de visualizar las cosas (código, errores, ejecuciones, etc.) según la tarea que se esté llevando a cabo: edición de código, ejecución, depuración, etc.

Es un IDE ágil, con una comunidad enorme detrás, con lo cual es fácil de encontrar soluciones a errores comunes y sugerencias para problemas típicos o formas de trabajar, sin mencionar que tiene detrás el soporte de una megacorporación como IBM y varias otras empresas que donan tiempo y dinero para mantener el proyecto activo y a la vanguardia de las tecnologías soportadas. Tiene extensiones (plugins) propios (desarrollados por la misma “fundación eclipse”) o de terceros. Las extensiones son fáciles de instalar ya que tiene un administrador incorporado. Un poco hambriento de recursos y con una estética un poco recargada (a mi gusto), es muy configurable, flexible y sencillo de usar. Cabe destacar que en los últimos tiempos agregó de forma nativa (antes era por medio de terceros) un “dark mode” que descansa un poco los ojos si el usuario lo desea. Usa conceptos sencillos y fáciles de asociar a un proyecto típico de software. Presenta múltiples ayudas al programador, ya sea para generar proyectos nuevos como para la creación de componentes dentro de un proyecto. Tiene grandes capacidades de refactoring y de ayudas al desarrollador al momento de solucionar errores de compilación y ejecución.

NetBeans

NetBeans nació en el 1996, antes que Eclipse, y es probablemente el IDE más sencillo de todos. Hoy forma parte de la instalación del entorno de desarrollo de JAVA (si uno lo elige). Quien soporta NetBeans es la “Apache Foundation”, lo que le da también una

amplia y activa comunidad. Es de código abierto, al igual que eclipse, y, por tanto, es gratuito. Es un IDE más liviano que Eclipse, aunque un poco menos configurable. Tiene múltiples ayudas al programador para el desarrollo, ejecución y depuración. La estética no es su fuerte, aunque sí es posible ponerlo en “dark mode” para no causar cansancio en los ojos. Si bien no tiene tantas herramientas como el eclipse (compilación, ejecución, empaquetado), su ventaja es que se utiliza muchas de las herramientas estándar de JAVA o herramientas que son parte del ecosistema estándar de JAVA (eclipse maneja su propio motor de empaquetado, NetBeans utiliza Maven para el empaquetado). NetBeans no es realmente “políglota” aunque soporta Java, C/C++ y PHP con su respectiva pata web. Al ser un IDE sencillo, es muy fácil de entender y utilizar. Posee buena potencia en lo que refiere a refactoring, búsquedas y ayudas, aunque apenas inferior que el eclipse.

IntelliJ IDEA

Nació en el 2001, pero estuvo relegado muchos años, especialmente porque era pago, y frente a Eclipse, con todo lo que brinda desde ese momento, era poco preferido entre los desarrolladores. La ventaja, al ser pago, es que tiene soporte personal para problemas y pedidos de *features*. Asimismo, siempre supo ser un poco más “pesado” que el eclipse e incluso más hambriento de recursos. En los últimos años, ganó mucha tracción dado que la empresa que lo desarrolla decidió mejorarlo mucho en cuanto performance y lanzaron al mercado una versión gratuita limitada (tiene todas las características, pero no soporta WEB de forma nativa/directa). También decidieron cambiar el modelo de negocio y se puede acceder a una licencia por un precio alto en un primer pago, y luego una renovación anual bastante conveniente. Dan licencias gratuitas a estudiantes, organizaciones sin fines de lucro, etc. De estética simple, tiene una potencia que enseguida sorprende: las búsquedas en código y entre archivos están previamente indexadas, por lo que son prácticamente instantáneas; la inteligencia en las sugerencias de auto-completado y solución rápida de problemas es superior a la de eclipse; utiliza, como NetBeans, herramientas estándar pero están muy bien abstraídas detrás de facilidades que parecen ser propietarias (por ejemplo, utiliza Maven para build-tool automation, pero no es necesario estar familiarizado con la herramienta); mantiene un

modelo del proyecto (basado en Maven) al igual que el eclipse; tiene algunas utilidades muy interesantes en cuanto al refactoring. Debe notarse que a veces sufre de ser “demasiado inteligente” y algunas cosas sencillas se tornan un poco frustrantes, pero son las menos (es algo similar a lo que pasa con Microsoft Word, que a veces uno quiere hacer una simple lista numerada, y termina luchando con los márgenes o sangrías porque Word “decide” que queda mejor de otra manera). En cuanto a la estética, está más alineado con los estándares de usabilidad actuales (por ejemplo, todos los comandos o autocompletado se introducen con TAB, en vez de CTRL+espacio como el eclipse o NetBeans, aunque no está con lo último, como la nueva costumbre de introducir comandos por nombre, mediante una consola o barra rápida de acceso << VSCode, Atom, SublimeText >>). IDEA no es un IDE “poliglota”, aunque IntelliJ, la compañía detrás de IDEA, vende múltiples versiones de la misma “base”, con herramientas específicas para cada lenguaje (IntelliJ IDEA es para Java, PyCharm para Python, ReSharper para C Sharp, etc., etc.). Ofrecen una versión “ultimate” que soporta todos los lenguajes, pero admiten que no tiene lo “último” de cada uno de los productos específicos.